Giving AI Assistants a Reliable Notebook

How Agent Memory keeps track of long, multi-day work—in plain words and concrete math

Dorian Herle
herle.dorian@gmail.com
https://github.com/dorianherle/agent_memory

October 14, 2026

The short version

AI assistants are increasingly asked to do work that spans many sessions, such as building software or working through a multi-step mathematical derivation. Yet each new session starts with a blank mind. Existing fixes either dump the entire past transcript into the prompt (which buries key facts and explodes in cost) or *summarize and slice* old notes (which destroys exact equations and code constants).

We built **Agent Memory**, which works like a well-kept engineering notebook: it saves whole notes with Git file snapshots, hands the assistant a deterministic **1,200-character cheat sheet** at the start of every turn (with zero extra AI calls), folds completed steps together, and lets the assistant pull up uncut pages in a single lookup when needed. On the ICML 2026 *MemoryArena* physics benchmark (20 papers, 86 steps), it kept **100% of needed equations intact** and helped a strong AI model finish **all 20 multi-step problems (86/86 steps)**, compared with 7 out of 20 without memory.

1 The problem: a colleague who forgets every morning

Imagine a brilliant colleague who gets amnesia overnight. On Monday you explain the setup of a project and derive a key formula. On Wednesday you ask them to apply Monday's formula to a new case. They remember nothing.

That is how AI assistants work across sessions today. To bridge the gap, people attach an external memory system. The three standard approaches all break down on technical work:

- **Hand over the whole transcript (Long Context).** Nothing is thrown away, but after k sessions the prompt contains every dead end, wrong turn, and verbose tool log. Important definitions get drowned out, and token cost grows quadratically ($\mathcal{O}(k^2)$).
- **Keep short AI-written summaries (e.g., Mem0).** An auxiliary AI rewrites each session into short bullet points like *"established a locality rule for the amplitude."* In technical work, the exact number or sign ($\sigma = 4$, $c_2 = -1$) is the very thing that gets paraphrased away.
- **Search the transcript in fixed-size slices (Chunk RAG / BM25).** The transcript is chopped every 400–500 characters. A definition at the top of a page and the equation at the bottom end up in separate slices, so searching retrieves one without the other (Figure 1).

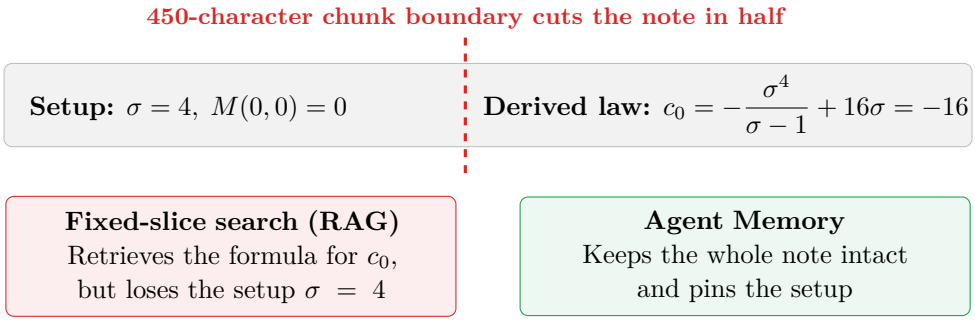


Figure 1: Why fixed-size chunking fails on math and code: slicing text every few hundred characters separates a formula from the definitions required to evaluate it.

2 How Agent Memory works: step-by-step schematic and concrete math

Scientists and engineers solved this problem long ago with a structured lab notebook: write complete entries, track when files change, read a one-paragraph status summary each morning, and flip to the full page when you need a detailed derivation. Agent Memory turns those habits into a deterministic six-step pipeline (Figure 2).

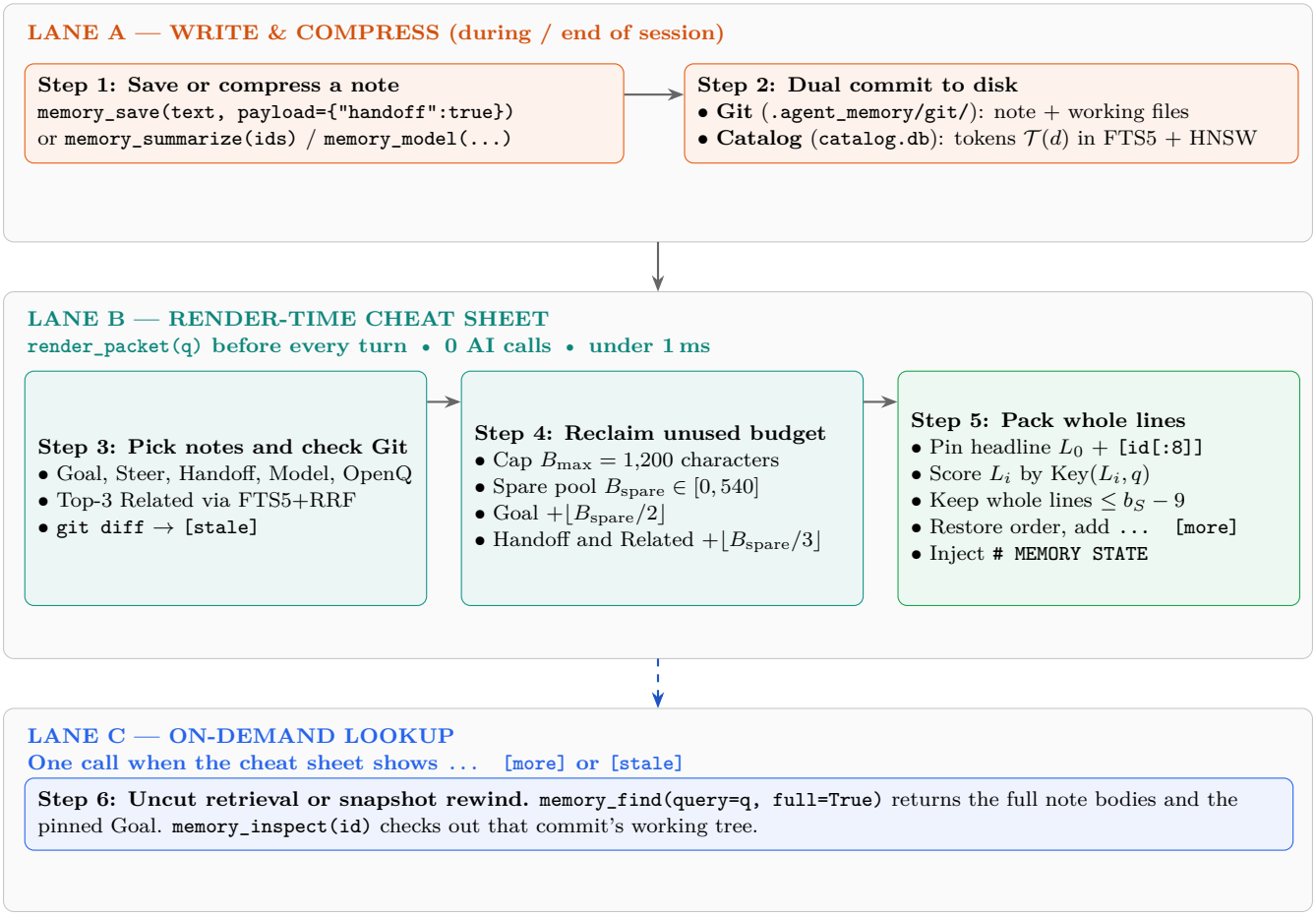


Figure 2: Complete six-step schematic block diagram of Agent Memory across saving (Lane A), deterministic render-time prompt compilation (Lane B), and on-demand full lookup (Lane C).

2.1 Mechanism 1: The 1,200-character morning cheat sheet (`render_packet`)

Before every turn, Agent Memory builds a compact block of text called `# MEMORY STATE` and places it directly in the assistant's prompt. No AI model is called to write or shorten this cheat sheet; it is compiled deterministically in under a millisecond.

Step 4 Math: How the 1,200-character budget is shared (Dynamic Budget Reclamation)

The cheat sheet has a strict total cap of $B_{\max} = 1{,}200$ characters ($\approx$ 200–300 tokens), with 54 characters reserved for section headers and omission notices. It is divided across six sections with base character allowances:

$$(b^0_{\text{Goal}}, b^0_{\text{Steer}}, b^0_{\text{Model}}, b^0_{\text{Handoff}}, b^0_{\text{OpenQ}}, b^0_{\text{Related}}) = (260, 220, 200, 360, 120, 260) \text{ chars.}$$

Whenever optional sections (`Steering`, `Task model`, or `Open questions`) are empty, Agent Memory collects their unused quotas into a spare pool $B_{\text{spare}} \in [0, 540]$:

$$B_{\text{spare}} = 220\,\mathbb{I}(\text{Steer} = \emptyset) + 200\,\mathbb{I}(\text{Model} = \emptyset) + 120\,\mathbb{I}(\text{OpenQ} = \emptyset),$$

and automatically redistributes that spare space to the three core reasoning sections:

$$b_{\text{Goal}} = 260 + \left\lfloor \frac{B_{\text{spare}}}{2} \right\rfloor, \qquad b_{\text{Handoff}} = 360 + \left\lfloor \frac{B_{\text{spare}}}{3} \right\rfloor, \qquad b_{\text{Related}} = 260 + \left\lfloor \frac{B_{\text{spare}}}{3} \right\rfloor.$$

In plain words: when there are no open questions or steering rules ($B_{\text{spare}} = 540$), the `Goal` budget automatically doubles from 260 to 530 characters, and `Handoff` grows from 360 to 540 characters—keeping foundational equations and handoff instructions from being cut off.

2.2 How Handoff and deterministic render-time line selection work

A critical question in multi-session work is: *how does the next session know exactly where the previous session left off, and how are lines selected when a note is longer than its character allowance?*

Part A: Which note becomes `## Handoff`? (The 3-Tier Priority Ladder) At render time, `_handoff(items)` selects a single transfer note using a strict three-tier priority ladder:

1. **Tier 1 — Explicit transfer checkpoint (`payload={"handoff": true}`):** If the previous session explicitly saved a handoff checkpoint, always select the newest one.
2. **Tier 2 — Newest actionable or uncovered finding:** Otherwise, scan backward through saved work notes and select the newest note that either (a) contains an explicit `Next:`, `Blocked:`, or `Remaining:` line, or (b) has not yet been absorbed into a verified `Task model`.
3. **Tier 3 — Latest milestone roll-up (`memory_summarize`):** If all recent steps have been folded into a milestone summary (`payload["covers"]`), select the newest summary roll-up.

Part B: At render time, how does the tool select lines deterministically? (`_excerpt`) Once a note is chosen for `Goal`, `Handoff`, or `Related` with character allowance b_S, `_excerpt(item, query, limit=`b_S`)` selects complete lines using a five-step deterministic algorithm—never calling an LLM and never slicing a line, command, or equation in half:

Step 5 Math: Deterministic Whole-Line Scoring & Knapsack Selection (`_excerpt`)

1. **Split into whole non-empty lines:** Split the note into stripped lines $[L_0, L_1, \ldots, L_{m-1}]$.
2. **Pin the headline (L_0) and reserve the omission badge:** Always include the title line L_0 (capped at $\min(100, b_S - 1)$ chars) at index 0, prefixed by `[id[:8]]` and `[stale]` if any tracked file changed. Reserve 9 characters at the end of the budget ($\text{limit}_{\text{body}} = b_S - 9$) for the suffix `" ... [more]"`.
3. **Score every body line L_i ($i \geq 1$) against the current query q:** Each line receives a four-part lexicographic priority tuple:

$$\text{Key}(L_i, q) = \Big(\underbrace{\mathbb{I}_{\text{priority}}(L_i)}_{\in\{0,1\}}, \underbrace{|\mathcal{T}(L_i) \cap \mathcal{T}(q)|}_{\in\{0,1,2,\ldots\}}, \underbrace{\mathbb{I}_{\text{eq}}(L_i)}_{\in\{0,1\}}, \underbrace{-i}_{\text{line order}} \Big),$$

   where:

   - $\mathbb{I}_{\text{priority}}(L_i) = 1$ if L_i starts with an actionable or conclusion prefix (`Next:`, `Blocked:`, `Remaining:`, `Decision:`, `Rejected:`, `Rollback:`, `Constraint:`, `Evidence:`, `Verified:`, `Result:`, `Final:`, `Law:`, `Formula:`, `Invariant:`, `Do not`, `Never`, `Keep`) or contains `\boxed`.
   - $|\mathcal{T}(L_i) \cap \mathcal{T}(q)|$ is the number of non-stopword tokens (`[\w-]+`) shared between line L_i and the current user question q.
   - $\mathbb{I}_{\text{eq}}(L_i) = 1$ if L_i contains a mathematical operator or relation (`=`, `<=`, `>=`, `\equiv`, `\propto`, `\implies`, `\iff`, `\frac`, `\sum`, `\int`).
   - $-i$ breaks ties in favor of earlier lines in the note.
4. **Greedy whole-line knapsack packing:** Sort body lines from highest $\text{Key}(L_i, q)$ to lowest. Walk the sorted list: if line L_i has not been seen and fits within the remaining budget ($\text{used} + \text{len}(L_i) + 1 \leq \text{limit}_{\text{body}}$), add (i, L_i) to chosen and advance used. *If a long line does not fit, skip it and continue testing shorter high-priority lines below it.*
5. **Restore original reading order & flag omissions:** Sort chosen back by original line index i ($0 < i_1 < i_2 < \ldots$) so the excerpt reads naturally from top to bottom. If any body line L_i was left out, append `" ... [more]"` so the agent knows it can call `memory_find(ids=["..."], full=True)`.

2.3 Worked toy example: a real Handoff across sessions

To see exactly how this works in practice, suppose an engineer uses an AI assistant across two sessions to fix a double-charge payment bug in `src/checkout.py`.

1. End of Session 1 (Saving the Handoff checkpoint): Before closing Session 1, Agent A saves a 7-line handoff note (`id = "8f3a19c2"`) linked to `files=["src/checkout.py"]`:

```
L0 (56 chars):  Checkout double-charge root cause and webhook fix plan
L1 (74 chars):  Investigated Stripe logs from Tuesday afternoon across 14 staging accounts.
L2 (71 chars):  Root cause:  webhook retry arrives before charge_id commit finishes.
L3 (62 chars):  Invariant:  count(charge_id) <= 1 for every (order_id, event_id)
L4 (69 chars):  Rejected:  Redis TTL lock (drops valid retries if worker crashes mid-tx).
L5 (68 chars):  Verified:  Postgres UNIQUE(order_id, idempotency_key) blocks duplicates.
L6 (66 chars):  Next:  wire idempotency_key into src/checkout.py charge_card() call.
```

The full 7-line note is 472 characters long. Suppose in Session 2 the `Handoff` section has an allowance of $b_{\text{Handoff}} = 310$ characters (so $\text{limit}_{\text{body}} = 310 - 9 = 301$ characters after reserving 9 chars for `" ... [more]"`).

2. Start of Session 2 (Deterministic line scoring & packing): The user opens a fresh session and asks:

> *"Query q: Wire the `idempotency_key` check into `src/checkout.py`."*
> Non-stopword query tokens: $\mathcal{T}(q) = \{$`wire`, `idempotency_key`, `check`, `src`, `checkout`, `py`$\}$.

At render time, `_excerpt` pins headline L_0 (56 chars used) and scores lines $L_1 \dots L_6$:

Line	Line Preview	Len	$\mathbb{I}_{\text{priority}}$	Overlap	$\mathbb{I}_{\text{eq}}$	Tuple Key	Knapsack Decision ($\text{limit}_{\text{body}} = 301$)
L_0	`Checkout double-charge root...`	56	*(Headline — always pinned at index 0)*				**KEEP** (used = 56)
L_6	`Next:  wire idempotency_key...`	66	1 (`Next:`)	5	0	$(1, 5, 0, -6)$	**#1 KEEP** $(56 + 67 = 123 \leq 301)$
L_5	`Verified:  Postgres UNIQUE...`	68	1 (`Verified:`)	1	0	$(1, 1, 0, -5)$	**#2 KEEP** $(123 + 69 = 192 \leq 301)$
L_3	`Invariant:  count(charge_id)...`	62	1 (`Invariant:`)	0	1 (`<=`)	$(1, 0, 1, -3)$	**#3 KEEP** $(192 + 63 = 255 \leq 301)$
L_4	`Rejected:  Redis TTL lock...`	69	1 (`Rejected:`)	0	0	$(1, 0, 0, -4)$	**#4 SKIP** $(255 + 70 = 325 > 301)$
L_2	`Root cause:  webhook retry...`	71	0	0	0	$(0, 0, 0, -2)$	**#5 SKIP** $(255 + 72 = 327 > 301)$
L_1	`Investigated Stripe logs...`	74	0	0	0	$(0, 0, 0, -1)$	**#6 SKIP** $(255 + 75 = 330 > 301)$

3. Final rendered `## Handoff` injected into Session 2: After step 5 re-sorts the kept lines (L_0, L_3, L_5, L_6) back into their natural top-to-bottom order $(0 < 3 < 5 < 6)$ and appends `... [more]`, Session 2 sees:

```
## Handoff
[8f3a19c2] Checkout double-charge root cause and webhook fix plan
Invariant:  count(charge_id) <= 1 for every (order_id, event_id)
Verified:  Postgres UNIQUE(order_id, idempotency_key) blocks duplicates.
Next:  wire idempotency_key into src/checkout.py charge_card() call.  ...  [more]
```

Notice what happened without any AI summarizer:

- Chatty narration (L_1) and secondary background (L_2, L_4) were automatically left out to fit the budget.
- The mathematical invariant (L_3), the verified database constraint (L_5), and the exact next step (L_6) survived *character-for-character* in their original reading order.
- The `... [more]` badge tells Agent B that 3 lines were omitted and can be retrieved verbatim via `memory_find(ids=["8f3a19c2"], full=True)`.
- If a teammate edits `src/checkout.py` before Session 2 starts, Step 3's `git diff` check sees that `src/checkout.py` changed since commit `8f3a19c2` and renders `[8f3a19c2] [stale] Checkout double-charge...`, warning Agent B to re-check the file before editing!

2.4 Mechanism 2: Symbol-aware search & one-step full lookup (`memory_find`)

The 1,200-character cheat sheet is enough for short and medium problems, but a 12-step physics derivation cannot fit in one paragraph. When the assistant needs the complete details, it makes a single call: `memory_find(query=q, full=True)`.

While testing on *MemoryArena*, we found two reasons standard search tools failed on math problems—and fixed both:

How symbol-aware search works (Tokenizer + Hybrid Rank Fusion)

1. Stripping punctuation without losing single-letter math symbols. Suppose a question asks: *"Use the* $\mathfrak{so}(1,2)$ *commutators* $([D,H],[D,K],[K,H])$ *to simplify the Hamiltonian."* A naive search engine splits on spaces, producing the token `"([D,H],"` with the brackets and comma glued to the letters. In SQLite's full-text search (FTS5), quoting a token with punctuation forces an exact phrase search that fails to match `[D,H] = iH` in the saved note! Meanwhile, common words like `"the"` and `"to"` match every note at random. Agent Memory extracts clean word and symbol tokens with a regular expression `\w+` and removes English filler words ($\mathcal{S}_{\text{stop}} = \{\texttt{the}, \texttt{and}, \texttt{for}, \dots\}$) while *preserving single-letter mathematical operators* like D, H, K, s, t:

$$\mathcal{T}(q) \;=\; \Big\{\, w \in \texttt{re.findall(r"\w+"}, q) \;\Big|\; \text{lower}(w) \notin \mathcal{S}_{\text{stop}} \,\Big\}.$$

On the 12-step Calogero–Moser problem, this single fix raised equation recall on Step 3 from **29% to 100%**.

2. Combining exact words and meaning (Reciprocal Rank Fusion). Each note d is ranked by both exact keyword match ($\text{rank}_{\text{FTS5}}(d)$) and optional semantic embedding similarity ($\text{rank}_{\text{vec}}(d)$), combined via Reciprocal Rank Fusion:

$$\text{Score}_{\text{RRF}}(d) \;=\; \frac{1}{60 + \text{rank}_{\text{FTS5}}(d)} \;+\; \frac{1}{60 + \text{rank}_{\text{vec}}(d)}.$$

When called with `full=True`, Agent Memory returns the **entire, uncut text** of the top-ranked notes in one step—and always pins the foundational `Goal` setup note so the assistant never loses the problem's initial definitions.

2.5 Mechanism 3: Git-snapshot history & automatic staleness detection

Every time an assistant saves a note with `memory_save`, Agent Memory commits that note into a private Git repository (`.agent_memory/git/`) along with a snapshot of the project's working files. If a note depends on specific files (for example, `files=["src/checkout.py"]`), Agent Memory records the cryptographic SHA-256 hash of each file at the moment the note was written.

Before every later turn, it checks whether any declared file $f \in \mathcal{F}_{\text{note}}$ has changed on disk:

$$\text{IsStale(note)} \;=\; \exists\, f \in \mathcal{F}_{\text{note}} \quad \text{such that} \quad \text{SHA256}_{\text{saved}}(f) \neq \text{SHA256}_{\text{current}}(f).$$

If a file was edited, created, or deleted since the note was written, the cheat sheet automatically marks the note `[stale]` so the assistant knows to re-verify it. And because every note is a Git commit, the assistant can call `memory_inspect(id)` to temporarily rewind the project files to how they looked when that note was written.

2.6 Mechanism 4: Consecutive compression (Rolling summaries & algebraic rules)

As work continues across many sessions, Agent Memory keeps the notebook compact in two ways:

1. **Rolling step summaries (`memory_summarize`):** After finishing step t, the assistant folds the verified conclusions of steps $1 \dots t$ into one summary note. The morning cheat sheet shows this single roll-up in `Handoff` and hides the individual step notes (while keeping the full originals searchable in Git).
2. **Turning repeated examples into compact rules (`memory_model`):** When an assistant tests many specific cases of a system rule (for instance, which file changes require a code review, or which sensor serial numbers need a calibration offset), listing every test case wastes space. Agent Memory lets the assistant write a safe one-line formula (such as `2.75 if serial == 'M17' else 4.0`), verifies that it passes every recorded historical test case, and automatically simplifies redundant rules using Boolean algebra:

$$(A \wedge B) \vee (A \wedge C) \longrightarrow A \wedge (B \vee C) \qquad \text{(factor out shared conditions)}$$

$$(x = a_1) \vee (x = a_2) \vee (x = a_3) \longrightarrow x \in [a_1, a_2, a_3] \qquad \text{(collapse repeated checks)}$$

3 How we tested it: the ICML 2026 MemoryArena benchmark

We evaluated Agent Memory on *MemoryArena* (`formal_reasoning_phys`), a peer-reviewed ICML 2026 benchmark created specifically to test whether AI agents can remember exact information across sessions. It consists of **20 theoretical-physics papers** broken into **86 sequential steps** (chains of 2 to 12 sessions per paper).

- **Session 1 ($t = 1$)** gives the full mathematical setup (`BACKGROUND`) and asks the first question.
- **Sessions $2 \dots K$ ($t \geq 2$)** start in a **completely fresh session** where the setup is omitted (`"No information provided."`). To solve Step t, the assistant must recall both the definitions from Session 1 and the intermediate formulas it derived in Sessions $1 \dots t-1$.

Following the official benchmark code (`eval.py`), we report two scores across all 20 papers:

$$\text{PS} = \frac{1}{20} \sum_{p=1}^{20} \frac{\text{steps solved in paper } p}{K_p}, \qquad \text{SR} = \frac{1}{20} \sum_{p=1}^{20} \mathbb{I}(\text{final step } K_p \text{ solved}).$$

4 What we found

Finding 1: 100% of the needed equations survive

Before running an AI solver, we checked all 66 dependent steps ($t \geq 2$) across the 20 papers and measured what fraction of the ground-truth LaTeX equations from earlier steps actually survived in the prompt shown to the assistant (Figure 3).

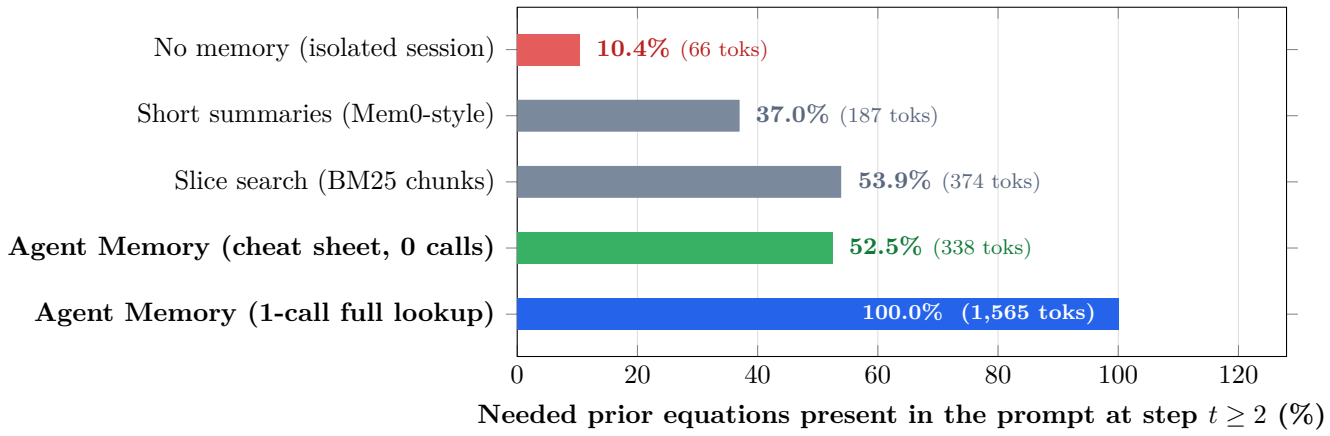


Figure 3: Equation retention across all 20 physics papers (66 dependent subtasks). Short summaries lose nearly two-thirds of the math (37.0% kept). Fixed-slice search loses nearly half (53.9%). Agent Memory's 1,200-character cheat sheet matches slice search with *zero* lookups (52.5%, green bar), and adding a single full lookup keeps **100.0%** of needed equations (blue bar).

Finding 2: On the official benchmark model (`gpt-5-mini`), Agent Memory ranks #1

To make a strict apples-to-apples comparison against the published ICML 2026 leaderboard, we ran all 20 papers using the benchmark's exact official model (`gpt-5-mini`) and official LLM equivalence judge (`MathEnvironment.judge()`).

On Step 1 alone—before any memory system is even used—`gpt-5-mini` only solves **12 out of 20 problems (60%)** due to single-step algebra errors. That 12-problem mark is the ceiling of what a perfect memory system could preserve for this model (Figure 4).

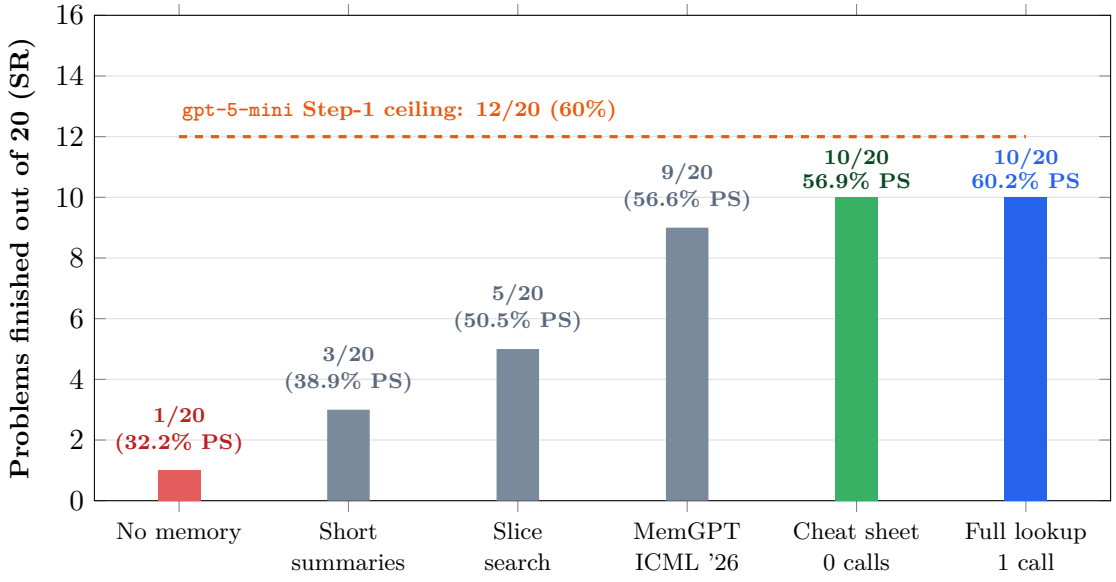


Figure 4: Results across all 20 papers using the official `gpt-5-mini` solver + LLM judge. Without memory, performance collapses from 12/20 on Step 1 down to 1/20 at the end. Agent Memory finishes **10 of the 12 solvable problems (50.0% SR; 56.9% PS for cheat sheet only in green, 60.2% PS for full lookup in blue)**, beating all published external memory systems.

Finding 3: With a stronger AI (`gpt-5.6-sol`), Agent Memory finishes 20 out of 20

To separate memory failures from algebra mistakes, we ran all 20 papers (86 steps) with a stronger solver (`gpt-5.6-sol`) that gets **all 20 first steps right (100%)**. Now any failure on Steps 2 . . . 12 is purely a memory failure (Figure 5).

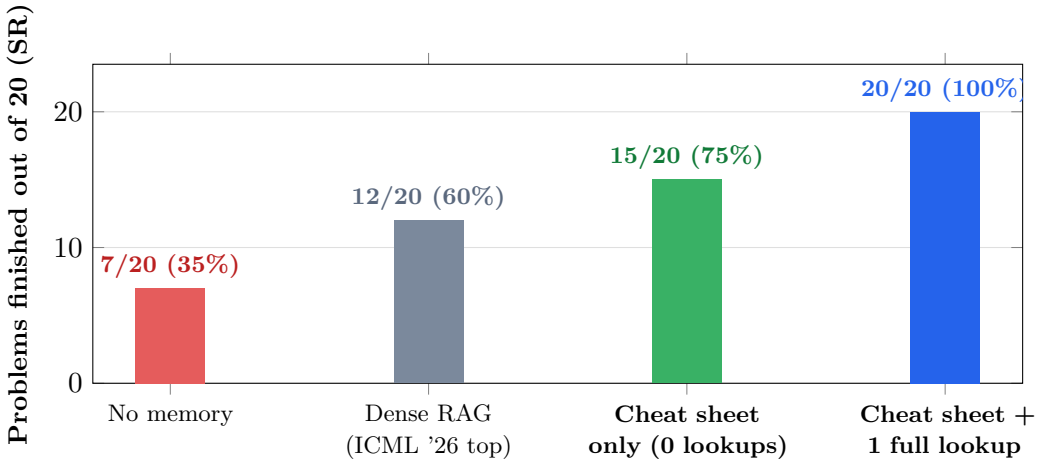


Figure 5: With `gpt-5.6-sol`, the 1,200-character cheat sheet alone finishes **15 of 20 papers (75.0% SR, 85.8% PS, green)** with zero tool calls ($\sim$ 197 tokens/turn), and adding one full lookup finishes **all 20 papers (100.0% SR, 86/86 subtasks solved, blue)**.

Finding 4: The longer the chain, the bigger the gap

Figure 6 tracks accuracy step by step from Session 1 to Session 12. Without memory, even the strong model drops from 100% at Step 1 to 45% at Step 2 and 0% after Step 6. The one-paragraph cheat sheet holds up well through Step 8 (66.7%), and the one-request full lookup stays at **100% all the way to Step 12**.

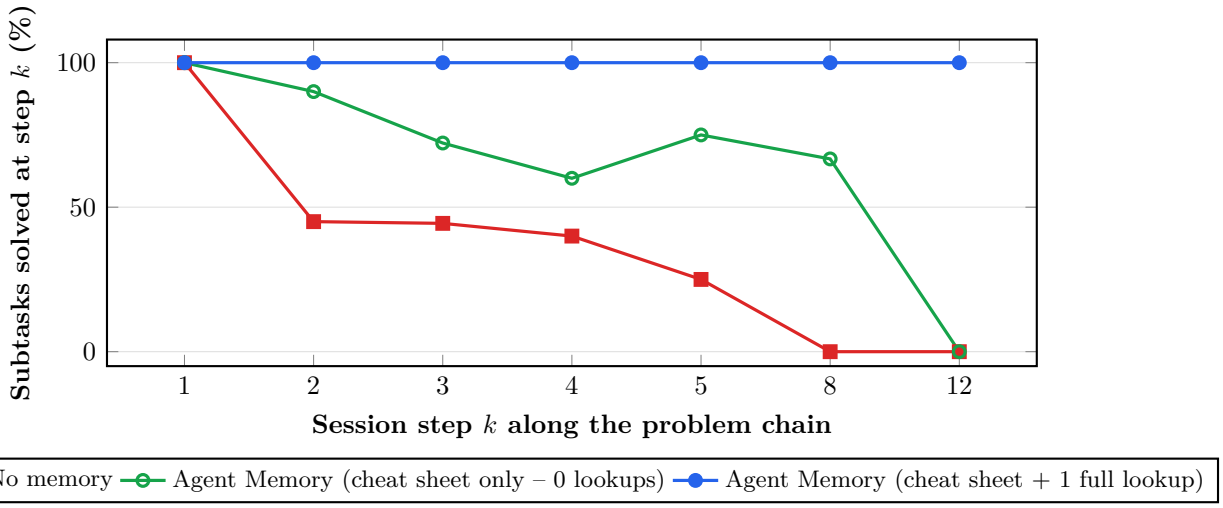


Figure 6: Accuracy by session depth k with `gpt-5.6-sol`. Without memory (red) accuracy collapses as chains get deeper; the 1,200-char cheat sheet (green) carries the agent through medium chains; one full lookup (blue) prevents decay through Step 12.

5 What this does and does not show

To keep our claims honest:

- **Evaluated on formal physics derivations.** Our live benchmark covers all 20 papers (86 subtasks) of MemoryArena's physics split (`formal_reasoning_phys`). While our adapter plugs directly into the other MemoryArena splits (math, CS, shopping, travel, search), we have only run the physics split so far.
- **Why we report two AI models.** On the benchmark's standard model (`gpt-5-mini`), Agent Memory achieves **50.0% SR and 60.2% PS**—beating MemGPT (45% SR), Mem0 (25% SR), and GPT-5.1-mini Long Context (45% SR / 60% PS), while sitting right at `gpt-5-mini`'s 60% single-step math ceiling. Running `gpt-5.6-sol` (100% SR / 100% PS) shows what happens when single-step algebra errors are removed.
- **Deep chains need the full lookup.** A 1,200-character paragraph ($\sim$ 200 tokens) solves 75% of papers with zero tool calls, but chains of 8–12 equations require calling `memory_find(..., full=True)` ($\sim$ 968–1,435 tokens).

6 Conclusion

Memory for AI assistants does not need to rely on lossy AI summaries or arbitrary text slicing. By saving complete notes as Git snapshots, compiling a deterministic 1,200-character cheat sheet every turn, folding finished steps into consecutive summaries or algebraic rules, and supporting one-call symbol-aware full lookup, Agent Memory preserves 100% of the underlying math and lets capable agents finish long, multi-session tasks without losing the thread.

Glossary. Agent: an AI assistant that works on a task across many steps and tool calls. **Session:** one work period; each new session starts with a fresh context window. **Long context:** feeding the entire past conversation transcript into every prompt. **RAG (Retrieval-Augmented Generation):** slicing past text into chunks and searching for matches. **FTS5 & RRF:** SQLite's fast full-text keyword search, combined with semantic search via Reciprocal Rank Fusion. **Git & SHA-256:** version-control snapshots and cryptographic file fingerprints used to detect when files change.